

UNIVERSITÀ
DEGLI STUDI
DI PADOVA

LAUREA TRIENNALE IN INFORMATICA

CORSO DI INGEGNERIA DEL SOFTWARE

ANNO ACCADEMICO 2026/2027



GRUPPO: 13oolean

EMAIL: gruppo13oolean@gmail.com

Valutazione dei capitolati

29-10-2026

Tabella 1: Registro delle revisioni del documento

Versione	Data	Autore	Descrizione delle modifiche	Verificato da
0.3.1	29-10-2026	Francesco	Correzione refusi	Tutti
0.3	29-10-2026	Gabriele	Stesura degli altri capitolati	Tutti
0.2	28-10-2026	Michele	Stesura dei capitolati selezionati	Tutti
0.1.1	27-10-2026	Michele	Stesura del capitolato scelto	Tutti

Indice

1 Introduzione **4**

2 Valutazione del capitolato selezionato **4**
2.1 C0: AIRA . . . 4
2.1.1 Descrizione . . . 4
2.1.2 Dominio . . . 4
2.1.3 Motivazione della scelta . . . 4
2.1.4 Conclusioni . . . 5

3 Valutazioni dei capitolati selezionati **5**
3.1 C8: Sonar . . . 5
3.1.1 Descrizione . . . 5
3.1.2 Dominio . . . 5
3.1.3 Motivazione della scelta . . . 5
3.1.4 Criticità riscontrate . . . 6
3.1.5 Conclusioni . . . 6
3.2 C2: NEXUM Concierge . . . 6
3.2.1 Descrizione . . . 6
3.2.2 Dominio . . . 6
3.2.3 Motivazione della scelta . . . 6
3.2.4 Criticità riscontrate . . . 7
3.2.5 Conclusioni . . . 7

4 Valutazione degli altri capitolati **7**
4.1 C1: AURA: AI for Understanding, Reasoning and Automation . . . 7
4.1.1 Pro . . . 7
4.1.2 Contro . . . 8
4.1.3 Conclusioni . . . 8
4.2 C3: FossDot: Smart speaker open source, estensibile e rispettoso della privacy . . . 8
4.2.1 Pro . . . 8
4.2.2 Contro . . . 8
4.2.3 Conclusioni . . . 9
4.3 C4: Piattaforma per la gestione dei veicoli e dei servizi post-vendita . . . 9
4.3.1 Pro . . . 9
4.3.2 Contro . . . 9
4.3.3 Conclusioni . . . 10
4.4 C5: RareCare: Piattaforma AI per la raccolta dati e il supporto decisionale nelle malattie rare . . . 10
4.4.1 Pro . . . 10
4.4.2 Contro . . . 10
4.4.3 Conclusioni . . . 10
4.5 C6: Sviluppare un Sistema AI di Caching Semantico e Dashboard FinOps . . . 10
4.5.1 Pro . . . 11
4.5.2 Contro . . . 11
4.5.3 Conclusioni . . . 11
4.6 C7: SmartDashboard: piattaforma di Business Intelligence potenziata da AI . . . 11
4.6.1 Pro . . . 11
4.6.2 Contro . . . 12
4.6.3 Conclusioni . . . 12

4.7 C9: SpecCheck: Sviluppo di Skills per OpenCode per la verifica della tracciabilità end-to-end 12
4.7.1 Pro 12
4.7.2 Contro 12
4.7.3 Conclusioni 13

1 Introduzione

In questo documento viene presentata l'analisi dei capitolati d'appalto che abbiamo svolto al fine di valutare la fattibilità e scegliere il più adatto per il gruppo di lavoro **13oolean**. Verranno esposte in maniera chiara le motivazioni alla base della scelta di candidarci per il capitolato **AIRA**, e altrettanto verranno analizzate le altre due proposte che avevamo fortemente considerato, **NEXUM Concierge** e **Sonar**. Verranno inoltre elencati i pro e i contro di ciascuno dei capitolati rimanenti per argomentare la decisione di averli scartati.

2 Valutazione del capitolato selezionato

2.1 C0: AIRA

2.1.1 Descrizione

Il capitolato richiede la progettazione di un sistema multimodale basato su intelligenza artificiale per l'elaborazione di flussi informativi eterogenei e non strutturati (e-mail, file allegati, tracciati vocali e immagini). Il sistema estrae e formalizza i dati in strutture standard (JSON o record di database), avviando automatismi di notifica ed esecuzione tramite API REST verso software gestionali aziendali (ERP, CRM, piattaforme B2B e sistemi di ticketing) o flussi e-mail.

2.1.2 Dominio

Dominio tecnologico: Durante l'incontro conoscitivo con il proponente è emerso che l'adozione di Java e Angular costituisce un requisito primario per permettere il futuro inserimento dell'applicazione nell'infrastruttura aziendale. Lo stack delineato prevede:

- Motore di elaborazione in Java con framework Spring Boot, architettura a eventi e scheduler multicanale;
- Interfaccia di gestione sviluppata in Angular e TypeScript;
- Orchestrazione semantica e analisi del testo affidate a LangChain4j e modelli LLM;
- Componenti di trascrizione audio (Whisper, Google Speech) e riconoscimento OCR (Tesseract, EasyOCR);
- Persistenza basata su RDBMS (PostgreSQL, MariaDB o SQLite) e configurazioni di sistema via JSON/YAML.

Tra i requisiti chiave figurano inoltre la modularità e la scalabilità dell'architettura, oltre ad un pannello per la configurazione dei canali e alla consultazione dei log.

Dominio applicativo: Il progetto opera nel contesto B2B, caratterizzato da comunicazioni e flussi informativi eterogenei, frammentati e spesso privi di formalizzazione standard (codici parziali, abbreviazioni o sintassi non convenzionali). Lo strumento mira a centralizzare l'acquisizione dei dati, automatizzando il parsing delle comunicazioni in ingresso (come e-mail o allegati) per estrarre informazioni strutturate e attivare le relative operazioni gestionali nei sistemi aziendali.

2.1.3 Motivazione della scelta

La decisione del gruppo si fonda sui seguenti fattori:

- **Interesse tecnico:** L'applicazione pratica dell'AI a un contesto industriale e l'integrazione di componenti eterogenee (LLM, OCR e trascrizione audio) rappresentano un forte stimolo per il gruppo.

- **Fattibilità operativa:** I requisiti primari appaiono ben delimitati nei tempi della didattica. La presenza di moduli opzionali (dashboard di reportistica e risposte automatiche) offre un margine per calibrare lo sviluppo in base al lavoro effettivo.
- **Supporto del proponente:** Sanmarco Informatica ha mostrato immediata disponibilità nel chiarire i dubbi emersi durante l'analisi preliminare, confermando il supporto di figure tecniche sia nell'avvio degli ambienti di sviluppo che nel superamento di eventuali blocchi metodologici.
- **Accessibilità dello stack:** Sebbene alcune componenti richiedano uno studio iniziale, il gruppo si affida a uno stack consolidato, ampiamente documentato e supportato da una vasta community. Questo riduce notevolmente il rischio tecnico legato all'apprendimento rispetto ad architetture più di nicchia.

2.1.4 Conclusioni

Visti i motivi sopra citati, il gruppo ha deciso di scegliere AIRA come progetto per cui candidarsi, ritenendolo il miglior equilibrio tra interesse, fattibilità e qualità del supporto offerto dal proponente.

3 Valutazioni dei capitolati selezionati

3.1 C8: Sonar

3.1.1 Descrizione

L'obiettivo del capitolato è lo sviluppo di un'applicazione web destinata al monitoraggio di store Shopify. L'applicativo ha il compito di mappare le componenti del negozio (app installate, campi personalizzati, webhook e codice dei temi), mettendone in luce dipendenze e ridondanze. La piattaforma deve generare documentazione navigabile ed esportabile in formato HTML o Markdown, producendo un report sulla presenza di codice morto o elementi orfani.

3.1.2 Dominio

Dominio tecnologico: Il capitolato prevede vincoli tecnologici ben definiti:

- Next.js (App Router) distribuito su Vercel, sfruttando Workflows e Cron Jobs per la gestione delle analisi in background;
- Supabase (PostgreSQL con politiche RLS) integrato esclusivamente con OAuth Google;
- TypeScript (modalità strict) e interfacce Admin GraphQL di Shopify.

Viene inoltre suggerita l'adozione di Tailwind CSS con shadcn/ui, React Flow o Cytoscape.js per l'albero delle dipendenze, insieme a suite di test quali Vitest e Playwright.

Dominio applicativo: L'ambito è quello dell'e-commerce ad alti volumi, dove l'accumulo nel tempo di estensioni e personalizzazioni rende complessa la tracciabilità delle modifiche. Lo strumento mira a supportare la manutenzione continua e le fasi di replatforming o passaggio di consegne.

3.1.3 Motivazione della scelta

Il capitolato ha suscitato forte interesse nel gruppo. Gli obiettivi chiari e lo stack tecnologico moderno (ottima opportunità di acquisizione competenze su strumenti molto richiesti) sono stati motivi che hanno portato alla sua selezione. Ha inciso positivamente inoltre la disponibilità dell'azienda in un affiancamento costante al fine di garantire il successo del percorso.

3.1.4 Criticità riscontrate

- **Carico di lavoro elevato:** Il capitolato richiede un impegno significativo a causa dei venti requisiti obbligatori da soddisfare, del vincolo di copertura del codice con test per almeno il 70% e della stesura di documentazione specifica. A questo si aggiunge la necessità di configurare manualmente uno store Shopify di test.
- **Elevata curva di apprendimento sulle tecnologie:** Nonostante lo stack tecnologico sia stato valutato positivamente, la necessità di apprendere un numero elevato di nuove tecnologie, contemporaneamente ad altri strumenti del dominio (ecosistema Shopify), costituirebbe un ostacolo troppo impegnativo per il gruppo.
- **Vincoli dei piani gratuiti:** L'impiego delle versioni free di Vercel e Supabase impone di monitorare costantemente le quote d'uso dei servizi cloud per evitare blocchi al workflow.

3.1.5 Conclusioni

In sintesi, le motivazioni riportate hanno reso C8 un candidato di punta per la scelta finale. Tuttavia, il peso complessivo dei numerosi requisiti obbligatori e l'elevato sforzo iniziale richiesto per lo studio delle tecnologie hanno portato il gruppo a ritenere l'opzione meno sostenibile rispetto al tempo a disposizione, preferendo orientarsi verso una proposta con un miglior equilibrio tra sfida tecnica e fattibilità temporale.

3.2 C2: NEXUM Concierge

3.2.1 Descrizione

Il progetto propone la realizzazione di un assistente conversazionale da integrare nella PWA della piattaforma NEXUM, orientato ai servizi HR e alla comunicazione interna. Il sistema deve rispondere in linguaggio naturale a quesiti su procedure e regolamenti aziendali, limitando il campo del prototipo iniziale (MVP) al modulo per la gestione di ferie e permessi.

3.2.2 Dominio

Dominio tecnologico: Il capitolato non impone vincoli rigidi sullo stack tecnologico per il Concierge. L'architettura deve integrarsi con l'infrastruttura esistente di NEXUM, mentre la scelta degli strumenti di AI, dei modelli linguistici, dell'infrastruttura RAG e dei database viene rinviata ad una fase di analisi preliminare da concordare con l'azienda.

Dominio applicativo: L'ambito è quello della gestione delle risorse umane (HR) e della comunicazione interna aziendale. Il sistema si inserisce in un contesto reale in cui i dipendenti interagiscono con la piattaforma per consultare regolamenti aziendali, verificare il proprio saldo ferie/permessi e inoltrare le relative richieste in modo guidato.

3.2.3 Motivazione della scelta

Il capitolato C2 ha suscitato un forte interesse nel gruppo grazie alla spiccata attualità delle tematiche proposte. La possibilità di lavorare con l'Intelligenza Artificiale generativa, applicando modelli LLM e architetture RAG a un contesto aziendale reale come quello della gestione delle risorse umane, ha rappresentato un elemento di notevole stimolo formativo per la crescita tecnica del gruppo.

3.2.4 Criticità riscontrate

- **Gestione della conoscenza documentale:** Il sistema deve garantire risposte tracciabili e citare le fonti, bloccando la generazione quando mancano i riscontri. Questo impone uno studio preventivo sui metodi di retrieval ed evaluation, ambito su cui il gruppo non ha ancora esperienza diretta.
- **Incertezza sui contratti di integrazione:** Le interfacce applicative per lo scambio dati (autenticazione, saldo ferie, scrittura su database) non sono ancora definite e verranno concordate solo a progetto avviato. L'assenza di API già pronte o documentate introduce un margine di incertezza sensibile nella stima delle prime fasi di lavoro.

3.2.5 Conclusioni

Il gruppo ha valutato l'azienda e il progetto C2 in modo estremamente positivo. Tuttavia, alla luce del bilancio complessivo tra i punti di forza e i fattori di incertezza individuati, si è scelto di non procedere con questo capitolato. Oltre alle criticità tecniche evidenziate, ha inciso sulla decisione finale l'elevato livello di concorrenza con altri gruppi di lavoro interessati alla medesima proposta, il che avrebbe introdotto un ulteriore elemento di incertezza sull'effettiva assegnazione. La scelta non deriva quindi da una mancanza di qualità del capitolato, ma dalla preferenza per un'altra proposta ritenuta maggiormente in linea con le aspirazioni del gruppo e con garanzie di assegnazione più elevate.

4 Valutazione degli altri capitolati

4.1 C1: AURA: AI for Understanding, Reasoning and Automation

Il capitolato proposto prevede la progettazione e la realizzazione di un'architettura cloud che sia in grado di interfacciare tramite MCP (Model Context Protocol) le API **KNX IoT** di un impianto domotico con i servizi commerciali di intelligenza artificiale (quali ad esempio Claude, ChatGPT e Gemini).

4.1.1 Pro

- **Uso di MCP e tecnologie moderne:** Questo progetto ha un alto potere formativo, richiedendo l'implementazione di tecnologie all'avanguardia e in rapida crescita come: l'uso del protocollo MCP, l'attuazione di pratiche di containerizzazione del codice tramite Docker, la predisposizione al cloud con AWS e ovviamente l'integrazione tra LLM e dispositivi IoT. Nel mondo lavorativo odierno la conoscenza della maggior parte di queste tecnologie rappresenta ormai un requisito minimo.
- **Obiettivi chiari e perimetro ben delimitato:** La proponente definisce in modo chiaro i macro-componenti obbligatori, definendo 17 casi d'uso base e 21 casi avanzati di interazione conversazionale. Questo abbassa di molto il rischio di burnout, evitando inoltre lo sviluppo di funzionalità out-of-scope.
- **Kit hardware fornito al gruppo:** La proponente mette a disposizione un kit hardware portatile comprensivo di gateway BLE, attuatori, comando tapparelle, termostato e sensori di presenza UWB. Questo, insieme agli obiettivi e i casi d'uso definiti in modo chiaro, permette di osservare in modo tangibile il reale funzionamento del sistema, oltre a rendere più semplici le fasi di test funzionale.

4.1.2 Contro

- **Complessità di compliance, accordi legali e dati sensibili:** Un impianto domotico tratta informazioni sensibili relative alla sfera privata e alla sicurezza dell'abitazione, come per esempio: rilevamento di presenza tramite sensori UWB, gestione ingressi e allarmi, statistiche di consumo energetico e simili. Il semplice trattamento "chiuso" di questi dati richiederebbe di per sé un accordo sul loro trattamento; in questo caso, si aggiungerebbe un ulteriore livello di complessità dovuto all'interazione degli LLM con questi dati.

4.1.3 Conclusioni

Fin dall'inizio abbiamo riscontrato grande interesse per questo progetto, suscitato soprattutto dall'utilizzo del protocollo MCP, la chiarezza degli obiettivi e la fornitura del kit hardware da parte della proponente, ma abbiamo deciso di orientarci verso altre proposte visto il grande onere richiesto per il trattamento dati e poiché abbiamo notato come il progetto fosse fin da subito conteso da molti gruppi.

4.2 C3: FossDot: Smart speaker open source, estensibile e rispettoso della privacy

Il capitolato proposto mira a realizzare un ecosistema interamente open source per uno smart speaker vocale. L'obiettivo principale è quello di tutelare la privacy dell'utente finale, dando pieno controllo del dispositivo.

4.2.1 Pro

- **Open Source e piena titolarità del codice:** È stato chiarito che l'intero ecosistema resterà di proprietà del gruppo con rilascio sotto licenza permissiva. Questo offre un'ottima opportunità di valorizzazione del portfolio personale dei membri del gruppo.
- **Tematiche attuali e tecnologie stimolanti:** Il progetto affronta concretamente i problemi della maggior parte degli smart speaker commerciali, come: vendor lock-in, la non chiara gestione della privacy domestica e l'obsolescenza programmata. Per fare ciò propone l'uso di tecnologie moderne quali Rust e modelli locali tramite Ollama, faster-whisper, Piper e l'uso di MCP.
- **Supporto tecnico formativo:** La proponente mette a disposizione sessioni di approfondimento tecnico con i propri software architect, oltre ad offrire supporto sulle pratiche di agentic coding, il che rende tale progetto altamente formativo.

4.2.2 Contro

- **Dispersione tra tecnologie hardware, embedded e software:** Il progetto richiede di lavorare su ambiti ingegneristici molto distanti tra loro, a partire dalla gestione dell'hardware (SBC, array di microfoni, amplificatore, vincoli fisici del contenitore e altri), lo sviluppo di un firmware a basso livello, l'implementazione di pipeline audio/AI in tempo reale e lo sviluppo di funzioni web di sincronizzazione tra le stanze.
- **Rischio di aumento dei tempi di sviluppo:** Non essendoci uno stack tecnologico imposto, il gruppo dovrà sceglierlo in modo autonomo; questo unito alla curva di apprendimento di Rust e alle incognite legate al collaudo dei componenti hardware, espone il gruppo a un forte rischio di dilatazione dei tempi di sviluppo e di dispersione soprattutto durante le prime fasi.

- **Complessità di testing e conformità GDPR:** Dimostrare il funzionamento del flusso completo su un prototipo funzionante rappresenta requisito obbligatorio ai fini di completamento del progetto, è richiesta infatti una copertura di test di almeno l'80% sulle componenti applicative. Un'ulteriore complessità viene aggiunta dalla gestione della voce, la quale rappresenta un dato sensibile e personale, che richiede particolare attenzione per quanto riguarda la conformità GDPR.

4.2.3 Conclusioni

Pur riconoscendo il valore etico del progetto, l'eccessiva frammentazione delle competenze necessarie e delle aree di sviluppo da coprire, è stata valutata dal gruppo come troppo dispersiva rispetto al tempo a disposizione.

4.3 C4: Piattaforma per la gestione dei veicoli e dei servizi post-vendita

Il capitolato proposto mira a realizzare una piattaforma destinata al settore automotive, con l'obiettivo di coordinare il ciclo di vita post-vendita del veicolo tra casa automobilistica, concessionarie, centri di assistenza e proprietari finali.

4.3.1 Pro

- **Eccellente stack tecnologico e moderno:** Viene proposto l'uso di tecnologie moderne e solide, che ad oggi sono fortemente richieste dal mercato: Node.js, NestJS, TypeScript e Go per il backend, Angular per il frontend, NATS o Apache Kafka per l'orchestrazione asincrona ad eventi, PostgreSQL, MongoDB e Redis per la persistenza dei dati e infine deployment su Kubernetes.
- **Indipendenza da dispositivi fisici:** Non è richiesta alcuna integrazione diretta con le centraline elettroniche dei veicoli, i dati telemetrici e di utilizzo possono essere infatti inseriti manualmente o generati tramite simulatore. Questo rimuove un importante livello di complessità.

4.3.2 Contro

- **Progetto estremamente orizzontale:** Il perimetro funzionale richiesto è vastissimo, prevede la realizzazione di tre portali web distinti: uno per la casa automobilistica, uno per la concessionaria e uno per il proprietario. È inoltre necessario coordinare almeno 5 microservizi applicativi: Modelli e Veicoli, Concessionarie e Proprietari, Manutenzioni e Appuntamenti, Campagne di Richiamo e Notifiche. A tutto ciò si sommano ben 16 requisiti funzionali. La mole di lavoro affinché ogni area sia bene integrata e funzionale, anche solo ai fini dell'MVP, espone il gruppo ad un elevato rischio di sovraccarico e burnout.
- **Complessità di compliance:** Vengono imposti requisiti stringenti di controllo degli accessi basato sui ruoli, cifratura dei dati in transito, e una rigorosa segregazione logica dei dati tra concessionarie diverse e tra singoli proprietari, il che necessita inoltre particolare attenzione per la tutela GDPR dei dati personali.
- **Onere gravoso di validazione e documentazione:** Vengono elencate 19 macro-categorie di test obbligatori e 7 categorie opzionali, oltre alla stesura di una documentazione tecnica particolarmente estesa.

4.3.3 Conclusioni

Sebbene lo stack tecnologico proposto sia tra i più interessanti e formativi, il gruppo ha ritenuto questo progetto troppo dispersivo: l'enorme quantità di funzionalità e test richiesti comporterebbe un carico di lavoro difficilmente sostenibile.

4.4 C5: RareCare: Piattaforma AI per la raccolta dati e il supporto decisionale nelle malattie rare

Il capitolato proposto mira a realizzare una piattaforma con assistente AI, che ha come obiettivo supportare le famiglie e i Care Navigator (figure che guidano i pazienti attraverso le complessità del sistema sanitario) nella gestione clinica, scolastica e burocratica delle malattie rare neurodegenerative, con particolare attenzione alla Sindrome di Sanfilippo.

4.4.1 Pro

- **Architettura di riferimento ben strutturata e impatto sociale:** La proponente fornisce una documentazione che delinea con estrema cura un'architettura Domain-Driven e Serverless su AWS, basata su un sistema plugin (Disease Pack), che permette di estendere il sistema a nuove patologie senza alterare il core applicativo. Il gruppo riconosce inoltre il grande impatto sociale che un tale progetto potrebbe avere.
- **Supporto e affiancamento tecnico aziendale:** La proponente garantisce affiancamento su cloud AWS, sull'integrazione AI, sul design UX/UI, e sulla mediazione dei requisiti.

4.4.2 Contro

- **Complessità normativa e di compliance:** Trattando dati sanitari e genetici, si è obbligati a rispettare linee guida severissime, come: redazione della valutazione d'impatto, vincolo di residenza dei dati in regioni UE, modelli di autorizzazione basati su ruoli, cifratura KMS AES-256 (con l'ulteriore complessità tecnica di conciliare la cifratura e l'elaborazione dei testi da parte dell'AI) con il rischio tra l'altro di ricadere nei vincoli del Regolamento sui Dispositivi Medici e dell'AI Act.
- **Delicatezza del dominio applicativo:** Trattando piani terapeutici, estrazione automatica di farmaci, dosaggi e sintomi, è richiesta tolleranza zero verso le allucinazioni del modello AI, questo implica l'uso di guardrail rigorosi, validazione delle fonti e revisione umana obbligatoria.
- **Vincolo sullo sviluppo di un'App Mobile:** Viene richiesto lo sviluppo di un'applicazione mobile e web cross-platform in Flutter o React Native, la quale, visto il campo di applicazione, necessita di essere conforme ai rigorosi criteri di accessibilità WCAG 2.1 AA e di design empatico.

4.4.3 Conclusioni

Pur apprezzando la qualità architetturale e soprattutto l'obiettivo sociale di questo progetto, l'estrema delicatezza del dominio di applicazione e il grande peso della compliance hanno portato il gruppo ad escludere questa proposta.

4.5 C6: Sviluppare un Sistema AI di Caching Semantico e Dashboard FinOps

Il capitolato proposto mira a realizzare un'architettura formata da due layer, che va ad estendere le funzionalità dell'orchestratore open source LiteLLM. Il primo è un layer di Caching Semantico, basato su embedding e ricerca per similarità, che mira a ottimizzare il consumo dei token AI, che

ad oggi, soprattutto in ambito aziendale, rappresentano una spesa non trascurabile. Il secondo layer consiste invece in una Dashboard FinOps finalizzata alla rendicontazione, monitoraggio e previsione dei costi legati al consumo dei token.

4.5.1 Pro

- **Stack tecnologico moderno e ben focalizzato:** Vengono proposte tecnologie moderne e ben perimetrate: Python 3.10+, LiteLLM, FastAPI/Flask, modelli di embedding come quelli di OpenAI, Redis, Vector Database su PostgreSQL, React con Chart.js e l'uso di Docker Compose.
- **Problema realistico e di forte rilevanza industriale:** Nel contesto odierno in cui l'uso dell'AI è centrale anche in ambito aziendale, l'ottimizzazione dei costi e l'eliminazione delle chiamate ridondanti rappresentano esigenze reali per le aziende.

4.5.2 Contro

- **Difficoltà nel generare un risultato realmente affidabile:** Sebbene l'obiettivo sia chiaro, implementare un meccanismo di caching semantico efficace in un contesto aziendale è estremamente complesso. Seppur alcune richieste possano risultare simili, potrebbero nascondere sfumature o contesti di sessione differenti, che rendono difficile riuscire a calibrare la soglia di similarità in modo da ridurre al minimo il rischio di falsi positivi, senza considerare gli effetti che tali risposte errate potrebbero generare in ambito aziendale.
- **Difficoltà nella separazione tra testing deterministico e probabilistico:** Viene richiesta una copertura dei test automatici di almeno il 70% e viene imposto di separare i test deterministici dalla valutazione probabilistica della cache semantica e dei modelli di previsione della spesa mensile, questa attività richiede metriche e dataset di validazione non banali da costruire in poco tempo.
- **Dipendenza da una codebase di terze parti:** Il vincolo di utilizzare LiteLLM come proxy obbliga il gruppo a investire tempo per uno studio profondo dell'architettura del progetto open source, al fine poi di innestarvi in modo pulito la nuova logica richiesta.

4.5.3 Conclusioni

Nonostante il capitolato presenti uno stack tecnico di grande interesse e affronti un problema molto stimolante e odierno, la complessità nel garantire l'affidabilità e il valore finale del risultato e la difficoltà di testing, hanno portato il gruppo a preferire altre alternative.

4.6 C7: SmartDashboard: piattaforma di Business Intelligence potenziata da AI

Il capitolato proposto mira a realizzare una piattaforma che permetta l'estrazione dei dati da un gestionale ERP, la trasformazione di essi tramite pipeline ETL in un Data Warehouse basato su DuckDB, e infine una piattaforma web per la composizione in stile drag & drop di dashboard analitiche affiancate da un agente AI.

4.6.1 Pro

- **Visione completa del ciclo di vita del dato:** Il progetto ruota attorno all'intero ciclo del dato, dall'ingestione nel Raw Data Store alle pipeline ETL in Python, fino ai database vettoriali e all'integrazione con LLM commerciali.

- **Dataset reale a disposizione:** La proponente offre la possibilità di lavorare su dataset aziendali reali, in particolare dei settori alimentare e logistico (ordini, vendite, magazzino), il che permette una validazione concreta delle performance.

4.6.2 Contro

- **Progetto estremamente orizzontale e vastità di perimetro:** Il progetto richiede di sviluppare un'architettura che accorpa tre sottosistemi molto onerosi: la pipeline ETL e un Data Warehouse, un costruttore di dashboard drag & drop comprensivo di griglia reattiva, filtri, funzioni di esportazione in PDF/Excel, vista mobile e 7 famiglie di widget complessi come gauge, mappe, tabelle pivot e altri, infine un layer AI capace di generare widget da testo libero, fare da assistente design, e altro ancora.
- **Eccessiva autonomia dell'utente e requisiti generici e flessibili:** Trattandosi di uno strumento self-service in cui l'utente ha piena autonomia, risulta estremamente complesso identificare e progettare tutte le funzioni e le query che il sistema dovrà supportare.
- **Estrema difficoltà di testing e validazione:** Le infinite combinazioni di widget, filtri e richieste conversazionali che ogni utente può effettuare, rendono il testing della piattaforma particolarmente arduo e poco affidabile.

4.6.3 Conclusioni

Per quanto l'obiettivo finale del progetto sia molto interessante e ambizioso, l'assenza di un perimetro definito e la difficoltà di testing che uno strumento self-service porta con sé, hanno portato il gruppo a scartare il capitolato, ritenendo reale il rischio di non riuscire a consolidare in modo concreto tutte le componenti richieste.

4.7 C9: SpecCheck: Sviluppo di Skills per OpenCode per la verifica della tracciabilità end-to-end

Il capitolato proposto mira alla realizzazione di un pacchetto integrato di estensioni come skill, tool, command, per l'agente OpenCode, con lo scopo finale di verificare automaticamente la tracciabilità e la conformità semantica tra casi d'uso, requisiti tecnici e codice sorgente di un progetto software esterno.

4.7.1 Pro

- **Ricerca innovativa:** Si esplora una problematica emergente di grande interesse nell'era dello sviluppo con AI, ovvero la verifica automatizzata della coerenza tra casi d'uso, requisiti tecnici e implementazione effettiva.
- **Fornitura dell'accesso LLM dalla proponente:** La proponente mette a disposizione sia l'accesso a modelli LLM proprietari enterprise, sia l'utilizzo illimitato di modelli open weight più compatti come Gemma 4 o Qwen 3.8 per testare la robustezza della soluzione.

4.7.2 Contro

- **Obiettivi finali poco chiari:** Trattandosi come detto prima, e come affermato anche dalla proponente, di un progetto a carattere fortemente esplorativo, questo lascia poco definiti i confini pratici e le metriche di accettazione del prodotto. Si rischia quindi di effettuare stime errate sui tempi di sviluppo oltre al rischio di sviluppo di funzionalità out-of-scope.

- **Complessità concettuale e algoritmica molto elevata:** Chiedere ad un LLM di interpretare diagrammi UML, dedurre la semantica di un intero codice sorgente, confrontarlo con le specifiche e rilevare codice non coperto da requisiti tecnici, vuol dire fargli toccare problemi teorici di alta complessità, facilmente esposti a falsi positivi e allucinazioni.
- **Frammentazione artefatti e progetto esterno da reperire:** Poiché non è possibile svolgere analisi deterministiche sui file dagli LLM, il capitolato impone di unire alle skill lo sviluppo di un Plugin per OpenCode o di un applicativo esterno con dashboard testuale, unificando il tutto tramite uno script di installazione poco stimolante dal punto di vista progettuale. Inoltre, è compito del gruppo reperire un intero progetto software esterno completo di casi d'uso, requisiti e codice su cui eseguire la validazione.

4.7.3 Conclusioni

Sebbene il progetto cerchi soluzione a una problematica di grande interesse nel mondo dello sviluppo odierno, a causa della scarsa definizione dell'obiettivo finale, dell'elevato rischio esplorativo e del poco interesse suscitato nel gruppo per certe implementazioni tecniche come il packaging degli script di installazione, il gruppo ha deciso di non candidarsi per questo capitolato.